N053 Cloud Run으로 무중단 배포하기_학습서

Chapter01

Cloud Run으로 무중단 배포하기

이 교안은 에이전트 서비스가 PR을 머지할 때 CI(코드를 합칠 때마다 테스트를 자동으로 돌리는 것)를 거쳐 GitHub Actions가 만든 이미지로 staging과 production에 배포되는 상태에서 시작합니다. 이 교안에서도 지금까지 배운 요소로 만든 AI 친구 에이전트 서비스를 예로 들어 실습하고, 줄여서 에이전트 서비스라고 부르겠습니다.

지금까지 여러 번 배포했지만, 배포하는 동안 사용자가 오류를 봤는지는 확인한 적이 없습니다. 새 버전이 준비되기 전에 요청을 받으면 어떻게 될까요. 새 버전을 사용자에게 보내기 전에 나만 먼저 볼 수는 없을까요. 일부 사용자에게만 먼저 보내고 숫자를 본 뒤 넓힐 수는 없을까요. 오늘은 Cloud Run이 리비전 사이에서 요청을 옮기는 방식을 배우고, 대화가 데이터베이스에 남는 에이전트 서비스로 블루그린과 카나리를 직접 해 본 뒤, 그 절차를 PR 머지 한 번으로 자동으로 돌게 만듭니다.

주관식 퀴즈

Q. [시작 질문] 새 버전을 배포했는데 사용자 100명 가운데 일부에게서만 답이 이상하게 나온다고 가정해 보세요. 한 번에 전환했을 때와 트래픽의 10%만 새 버전으로 보냈을 때, 문제를 발견하기까지와 피해 범위가 각각 어떻게 다를지 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. Cloud Run의 리비전, 트래픽 설정, 태그 URL, 스타트업 프로브가 무중단 배포에서 각각 무엇을 맡는지 설명할 수 있습니다.
2. staging과 production 서비스에 Supabase를 연결하고, secret key 를 시크릿으로 넣어 새 리비전에서도 대화가 남게 할 수 있습니다.
3. 배포하는 동안의 요청 실패를 측정하고, 준비가 늦은 버전의 실패를 `/health` 스타트업 프로브로 막을 수 있습니다.

4. 트래픽 0% 배포와 태그 URL로 블루그린 전환과 롤백을, 트래픽 비율로 카나리를 손으로 할 수 있고, 트래픽 고정 때문에 생기는 문제를 설명할 수 있습니다.
5. 0% 배포, 스모크 테스트, 전환, 롤백을 하나의 자동 배포 흐름으로 설계하고, 배포 중 실패 0건과 망가진 버전에서 멈추는 동작을 기록으로 증명할 수 있습니다.

Chapter02

로드밸런서를 Cloud Run이 맡는다

무중단 배포는 서비스 앞에서 요청을 받아 뒤의 서버로 나눠 보내는 로드밸런서(load balancer)가 있어야 할 수 있습니다. 이전 버전을 끄지 않은 채 새 버전을 띄우고, 로드밸런서가 요청을 보내는 곳을 새 버전으로 바꾼 뒤 이전 버전을 끄는 것이 기본 구조입니다. 서버를 직접 운영한다면 Nginx 같은 프로그램을 로드밸런서로 세우고, 설정 파일을 고쳐 보내는 곳을 바꿉니다.

Cloud Run에서는 이 로드밸런서를 Cloud Run이 운영합니다. 서비스 주소로 들어온 요청은 Cloud Run이 받아, 서비스의 리비전(revision, 배포할 때마다 만들어지는 이미지와 설정의 한 벌) 가운데 하나로 보냅니다. 어느 리비전에 요청의 몇 퍼센트를 보낼지 정한 것을 트래픽(traffic) 설정이라고 하고, 배포는 결국 새 리비전을 만들고 이 트래픽 설정을 바꾸는 일입니다. 아래 그림은 Nginx로 직접 구성할 때 사람이 하는 일을 Cloud Run에서는 무엇이 하는지 나란히 놓은 것입니다.

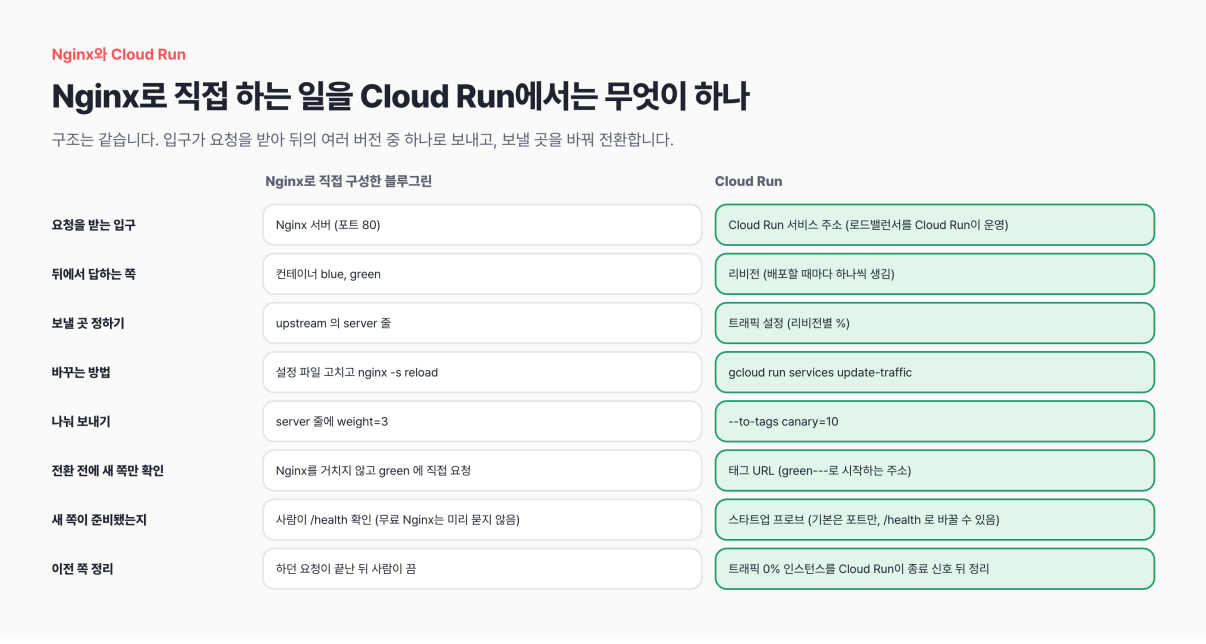


그림: Nginx로 직접 구성한 블루그린과 Cloud Run에서 같은 일을 하는 것

기본 배포가 하는 일

옵션 없이 배포하면 Cloud Run은 다음 순서로 움직입니다. 새 리비전의 인스턴스를 켜고, 스타트업 프로브(startup probe, 새 인스턴스가 요청을 받을 준비가 됐는지 확인하는 검사)를 통과하면 트래픽 100%를 새 리비전으로 옮기고, 이전 리비전의 인스턴스에는 종료 신호(SIGTERM)를 보낸 뒤 10초를 기다렸다가 끕니다. 이 10초 동안 처리 중이던 요청을 마치는 것이 그레이스풀 셧다운(graceful shutdown)입니다.

문제는 기본 스타트업 프로브가 "컨테이너의 포트가 열렸는지"만 본다는 점입니다. 앱이 포트는 바로 열었지만 모델 설정이나 색인을 불러오느라 아직 답할 준비가 안 됐다면, Cloud Run은 준비가 끝났다고 보고 트래픽을 옮기고, 그사이 들어온 요청은 실패합니다. 스타트업 프로브를 `/health` 처럼 앱이 준비된 뒤에야 정상 응답하는 주소로 바꾸면, 준비가 끝날 때까지 트래픽이 이전 리비전에 남습니다. 섹션 4에서 이 차이를 직접 측정합니다.

배포 방식을 Cloud Run 명령으로

블루그린과 카나리도 같은 도구로 합니다. 새 리비전을 트래픽 0%로 올리고(`--no-traffic`), 그 리비전에만 닿는 별도 주소를 붙인 뒤(`--tag`), 트래픽 설정을 바꿉니다. 태그를 붙이면 `https://green---friend-agent-staging-….run.app` 처럼 태그 이름으로 시작하는 주소가 생기는데, 이것을 태그 URL이라고 합니다. 태그 URL은 트래픽 설정과 상관없이 태그를 붙인 리비전 하나로만 요청을 보내는 별도 주소라서, 새 리비전이 트래픽 0%여도 이 주소로는 열어 볼 수 있습니다. 서비스가 공개되어 있으면 태그 URL도 누구나 열 수 있습니다.

방식	하는 일	명령
기본 배포	준비되면 바로 100%	`gcloud run services update …`
블루그린	0%로 올려 태그 URL로 확인한 뒤 한 번에 100%	`--no-traffic --tag green`, 그다음 `update-traffic --to-tags green=100`
카나리	0%로 올린 뒤 10%, 50%, 100%로 나눠 옮김	`update-traffic --to-tags canary=10` 을 비율을 바꿔 반복
롤백	트래픽을 이전 리비전으로 되돌림	`update-traffic --to-revisions <이전 리비전>=100`

롤백은 이미지를 다시 만들지 않고 트래픽 설정만 바꾸므로 몇 초 만에 끝납니다. 다만 Cloud Run은 롤백할 때 스타트업 프로브를 다시 돌리지 않고 데이터베이스 같은 바깥 의존성도 확인하지 않으므로, 되돌아갈 리비전이 지금도 정상인지는 사람이 확인합니다.

트래픽 고정

트래픽 설정에는 두 가지 모양이 있습니다. "가장 최근 리비전에 100%"로 되어 있으면 새 리비전이 배포될 때마다 트래픽이 따라 옮겨 가고, "리비전 3에 100%"처럼 특정 리비전이나 태그에 비율을 정해 두면 그 설정이 그대로 남습니다. 롤백이나 카나리를 한 번 하면 트래픽이 특정 리비전에 고정되고, 그 뒤의 배포는 새 리비전을 만들기만 하고 트래픽은 옮기지 않습니다. 배포는 성공으로 끝났는데 사용자는 여전히 이전 버전을 쓰는 상황이 이렇게 생깁니다. 고정을 풀려면 `update-traffic --to-latest` 로 다시 가장 최근 리비전을 따라가게 합니다.

움직여 보는 리비전과 트래픽

아래 시각화에서 점 하나는 요청 하나이고, 가운데 상자에는 그 순간의 트래픽 설정이 나옵니다. ① 탭에서는 시작 지연을 켜고 프로브를 "포트 열림"과 "/health 응답"으로 바꿔 가며 "새 리비전 배포"를 눌러, 503 실패가 생기는지 봅니다. ② 탭에서는 0% 배포 뒤 사용자 요청은 리비전 1로, 내 요청만 태그 URL로 리비전 2에 가는 모습과, 전환과 롤백을 봅니다. ③ 탭에서는 "새 버전에 오류 있음"을 켜고 10%에서 멈췄을 때 오류를 본 요청 수를 봅니다. ④ 탭에서는 롤백한 뒤 "새 리비전 배포"를 눌러 새 리비전이 트래픽을 받지 못하는 것과, `--to-latest` 로 풀리는 것을 봅니다.

객관식 퀴즈

Q. 카나리를 하다가 문제가 있어 `--to-revisions <이전 리비전>=100` 으로 롤백했습니다. 다음 날 버그를 고친 PR을 머지했고 배포 워크플로도 초록색으로 끝났는데, 서비스 주소의 `/version` 은 여전히 이전 리비전입니다. 가장 알맞은 설명은 무엇인가요?

• 1. 배포 워크플로가 이미지를 만들지 못했다
• 2. 롤백할 때 트래픽을 특정 리비전에 정해 두어, 새 리비전은 만들어졌지만 트래픽 0%로 남았다
• 3. 스타트업 프로브가 실패해 새 리비전이 지워졌다
• 4. 태그 URL이 서비스 주소를 대신하고 있다**제출**

- •

  1. 배포 워크플로가 이미지를 만들지 못했다

- •

  2. 롤백할 때 트래픽을 특정 리비전에 정해 두어, 새 리비전은 만들어졌지만 트래픽 0%로 남았다

- •

  3. 스타트업 프로브가 실패해 새 리비전이 지워졌다

- •

  4. 태그 URL이 서비스 주소를 대신하고 있다

📚 리비전과 트래픽 참고 자료

- Rollbacks, gradual rollouts, and traffic migration: 트래픽 0% 배포, 태그 URL, 단계적 전환, 롤백, 트래픽 고정을 한곳에서 확인할 수 있습니다.
- Configure container health checks: 스타트업 프로브의 기본값과 HTTP 프로브 설정 방법이 있습니다.
- Container runtime contract: 인스턴스를 끌 때 종료 신호를 보내고 기다리는 시간이 나옵니다.

Chapter03

시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 마지막 줄의 Supabase 시험용 프로젝트는 데이터베이스와 익명 로그인을 켜 둔 프로젝트 하나입니다. 없으면 템플릿의 `docs/supabase.md` 1단계로 이 섹션 안에서 만들어도 됩니다.

앞 실습을 이어서 하고 있다면 아래 확인 명령만 실행하고 넘어갑니다. 빠진 상태가 있으면 템플릿의 실습별 시작 상태 문서에서 "없으면" 칸의 항목을 표의 위 칸부터 차례로 실행합니다. 처음부터 모두 맞추면 한두 시간이 걸리므로, 빠진 항목이 많다면 수업 전에 맞춰 두거나 강사에게 먼저 알립니다.

필요한 상태	확인할 것	없으면
Google Cloud 프로젝트와 gcloud 설정, `friend-agent` 서비스	`gcloud config list`, `gcloud run services list` 에 `friend-agent`	A, B
배포 권한(WIF)과 저장소 변수	`gh variable list` 에 `GCP_PROJECT_ID`, `WIF_PROVIDER`, `DEPLOY_SA`, `RUNTIME_SA`, `AR_REPO`	`docs/wif-setup.md` 0~5, 7단계
CI와 환경별 배포, `staging` 브랜치, GitHub 환경, `friend-agent-staging` 서비스	`.github/workflows/ci.yml`, 원격 `staging` 브랜치, `gcloud run services list` 에 `friend-agent-staging`	C, D, E
루트의 `Dockerfile` 과 Actions에서 이미지를 만들	`deploy.yml` 안에 `docker build` 와 `image:` 줄	F, G

필요한 상태	확인할 것	없으면
어 배포하는 `deploy.yml`		
Supabase 시험용 프로젝트(테이블, 익명 로그인)	Supabase 대시보드에 프로젝트가 있고, Authentication에서 Allow anonymous sign-ins 가 켜져 있음	`docs/supabase.md` 1단계

직접 실행할 명령: 시작 상태 확인

```
gcloud config list
gcloud run services list --region us-central1
gh variable list
git branch -r
grep -n "docker build" .github/workflows/deploy.yml
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 의 표를 보고, 이 저장소와 지금 gcloud 프로젝트가 '블루그린과 카나리' 실습을 시작할 상태인지 확인해 줘.
확인 명령만 실행하고 아무것도 만들거나 바꾸지 마. 빠진 항목이 있으면 문서의 어느 항목(A~I)을 실행해야 하는지 알려 줘.
```

리비전이 바뀌어도 대화가 남으려면

지금 Cloud Run의 에이전트 서비스는 캐릭터, 대화, 기억을 인스턴스 메모리에 둡니다. 새 리비전이 뜨면 새 인스턴스가 시작하므로 그동안의 대화가 사라지고, 블루그린이나 카나리로 두 리비전이 함께 요청을 받으면 요청마다 보이는 대화가 달라집니다. 무중단 배포를 하기 전에 데이터를 모든 리비전이 함께 보는 데이터베이스로 옮겨 서비스를 무상태(stateless, 요청 처리에 필요한 상태를 인스턴스 안에 두지 않는 것)로 만듭니다.

데이터베이스는 Supabase를 씁니다. Supabase는 PostgreSQL 데이터베이스와 로그인 기능(Auth)을 함께 제공하는 관리형 서비스이고, 처음에는 가입 없이 브라우저마다 게스트 사용자를 하나씩 만드는 익명 로그인으로 시작합니다. staging은 시험용 프로젝트에, production은 새로 만드는 프로젝트에 연결해 시험하며 쌓인 데이터가 실제 사용자 데이터와 섞이지 않게 합니다. 무료 플랜은 활성 프로젝트가 2개까지이고, 1주 동안 활동이 없으면 일시정지됩니다(2026년 10월 기준).

직접 확인할 곳: production용 Supabase 프로젝트 만들기

1. Supabase 대시보드에서 `friend-agent-production` 프로젝트를 만듭니다. 리전은 Cloud Run 서비스가 있는 us-central1과 가까운 미국 리전(예: East US (Ohio))을 고릅니다. 리전이 멀면 요청마다 지연이 붙습니다.

2. SQL Editor 에 템플릿 `supabase/migrations/0001_init.sql` 의 내용을 붙여 넣고 실행합니다. 모든 테이블에 RLS(Row Level Security, 행 단위 접근 제한)가 켜져 브라우저는 테이블을 직접 읽거나 쓸 수 없습니다.
3. Authentication > Sign In / Providers 에서 Allow anonymous sign-ins 를 켜고 저장합니다. 꺼져 있으면 화면에 "로그인을 시작하지 못했습니다" 창이 뜹니다.
4. Authentication > Rate Limits 에서 익명 로그인 한도(기본 IP 하나당 시간당 30건)를 확인합니다.
5. 두 프로젝트 각각의 Project Settings > API Keys 에서 프로젝트 URL, publishable key, secret key 가 어디 있는지 확인합니다. 키 값은 채팅이나 명령, 코드에 붙여 넣지 않습니다.

키를 시크릿으로 넣기

Supabase에서 받은 값은 세 가지이고, 넣는 곳이 다릅니다. 프로젝트 URL과 publishable key 는 브라우저가 로그인할 때 쓰는 값이라 화면 코드로 전달되므로 일반 환경변수로 넣습니다. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터를 읽고 쓸 수 있는 키라서 서버에서만 쓰고, LLM API 키처럼 Secret Manager에 시크릿으로 넣습니다. staging과 production의 secret key 는 서로 다른 시크릿으로 두고, 런타임 서비스 계정이 두 시크릿을 하나씩 읽을 수 있게 합니다.

시크릿을 만드는 단계는 키 값이 명령 기록이나 에이전트 대화에 남지 않도록 콘솔에서 직접 합니다. Google Cloud 콘솔 > 보안 > Secret Manager > 보안 비밀 만들기에서 `supabase-secret-staging` 과 `supabase-secret-production` 을 만들고, 각각 그 환경 Supabase 프로젝트의 secret key 를 값으로 넣습니다. 그다음 읽기 권한은 이름만 오가므로 명령이나 에이전트로 줍니다.

직접 실행할 명령: 런타임 서비스 계정에 두 시크릿 읽기 권한 주기

```
gcloud secrets add-iam-policy-binding supabase-secret-staging --member "serviceAccoun
t:friend-agent-runtime@<PROJECT_ID>.iam.gserviceaccount.com" --role roles/secretmanage
r.secretAccessor
gcloud secrets add-iam-policy-binding supabase-secret-production --member "serviceAcco
unt:friend-agent-runtime@<PROJECT_ID>.iam.gserviceaccount.com" --role roles/secretmana
ger.secretAccessor
```

코딩 에이전트에게 맡길 때: 두 시크릿 읽기 권한

```
friend-agent-runtime 서비스 계정에 Secret Manager 시크릿 supabase-secret-staging 과 supabas
e-secret-production 각각에 대해서만 읽기 권한을 줘.
```

```
프로젝트 전체에 권한을 주지 말고, 시크릿 값은 읽거나 출력하지 마.
```

이 구성의 한계: 환경이 같은 런타임 서비스 계정을 쓴다

지금은 staging 서비스와 production 서비스가 같은 런타임 서비스 계정 `friend-agent-runtime` 으로 실행되고, 이 계정이 두 시크릿을 모두 읽을 수 있습니다. 그래서 staging 서비스도 production의 secret key 를 읽을 수 있는 상태입니다. staging은 시험 중인 코드가 먼저 올라가는 곳이라 실수나 취약점이 생기기 쉬운데, 이 구성에서는 staging에서 생긴 문제가 production 사용자의 데이터까지 닿을 수 있습니다. 데이터는 환경별로 나눴지만 권한은 나누지 않은 것입니다.

실무에서는 환경마다 런타임 서비스 계정을 따로 두고, 각 계정에는 그 환경의 시크릿만 읽게 합니다. 이 과정의 배포 워크플로는 저장소 변수 `RUNTIME_SA` 로 계정을 정하므로, GitHub의 staging 환경에 같은 이름의 변수를 두면 워크플로를 고치지 않고도 나눌 수 있습니다. 더 나아가면 배포 서비스 계정과 Google Cloud 프로젝트까지 환경별로 나눕니다. 이 교안에서는 실습 단계를 줄이려고 계정 하나로 진행하고, 이 차이는 서비스를 실제로 공개하기 전에 정리할 항목으로 남겨 둡니다.

staging에 연결하고 확인하기

staging 서비스에 다섯 가지 값을 넣습니다. `STORAGE=supabase` 와 `AUTH=supabase` 는 저장소와 로그인을 Supabase로 바꾸는 스위치이고, 나머지는 시험용 프로젝트의 URL, publishable key, secret key 시크릿입니다. 서비스에 넣은 환경변수와 시크릿 연결은 다음 자동 배포에서도 그대로 남습니다. 배포 워크플로는 커밋 SHA와 `ENV` 만 바꿔 넣고 나머지 설정은 서비스에 있는 값을 이어 쓰기 때문입니다.

직접 실행할 명령: staging 서비스에 Supabase 연결

```
gcloud run services update friend-agent-staging --region us-central1 \
  --update-env-vars STORAGE=supabase,AUTH=supabase,SUPABASE_URL=<시험용 프로젝트 URL>,SUP
ABASE_PUBLISHABLE_KEY=<시험용 publishable key> \
  --update-secrets SUPABASE_SECRET_KEY=supabase-secret-staging:latest
```

코딩 에이전트에게 맡길 때: staging 서비스에 Supabase 연결

```
friend-agent-staging 서비스에 환경변수 STORAGE=supabase, AUTH=supabase, SUPABASE_URL, SUPA
BASE_PUBLISHABLE_KEY 를 추가하고,
SUPABASE_SECRET_KEY 는 Secret Manager 의 supabase-secret-staging 최신 버전으로 연결해 줘.
```

```
URL과 publishable key 는 내가 직접 붙여 넣을 테니 자리만 비워 둔 명령을 보여 줘. 다른 환경변수와 설정은 바꾸지 마.
```

실행 전에 서비스 이름이 `friend-agent-staging` 인지, 시크릿이 `-staging` 쪽인지, 환경변수 옵션이 기존 값을 지우는 `--set-env-vars` 가 아니라 `--update-env-vars` 인지 봅니다. publishable key 는 공개되는 값이라 명령에 직접 넣어도 되지만, secret key 는 명령에 넣지 않고 시크릿 이름으로만 연결합니다. 이 교안에서 줄 끝에 `\` 를 붙여 여러 줄로 나눈 명령은 모두 macOS와 Linux 기준입니다. Windows PowerShell에서는 `\` 대신 백틱(`` ` ``)으로 줄을 잇거나 한 줄로 이어 씁니다.

직접 확인할 곳: staging에서 데이터가 남는지 확인

1. staging 주소를 열면 화면 왼쪽 아래에 "게스트로 사용 중"이 보입니다.
2. 캐릭터를 저장하고 메시지를 몇 개 보낸 뒤, Supabase Table Editor 의 `messages` 테이블에 방금 보낸 대화가 행으로 생겼는지 봅니다. 캐릭터를 저장하지 않으면 화면이 처음 열릴 때 캐릭터 만들기에서 멈추고 대화 기록을 불러오지 않습니다.
3. `gcloud run services update friend-agent-staging --region us-central1 --update-env-vars BANNER=저장확인` 으로 새 리비전을 하나 만듭니다. 새 리비전이 뜬 뒤 새로고침해도 대화가 그대로 남아 있는지 봅니다.
4. 다른 브라우저나 시크릿 창으로 staging 주소를 열면 다른 게스트 사용자로 시작해, 앞의 대화가 보이지 않는지 봅니다.

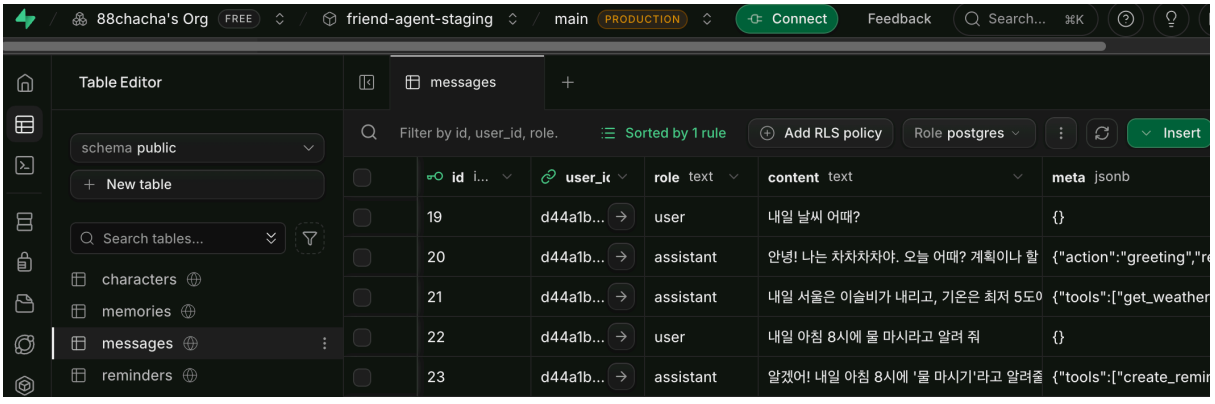


그림: Supabase 대시보드 > Table Editor > messages 에서 staging 화면에서 보낸 메세지가 user_id, role, content와 함께 행으로 쌓임

결과 해석: 리비전이 바뀌어도 이어지는 대화

새 리비전은 새 인스턴스로 시작했지만 대화는 그대로 보입니다. 대화가 인스턴스 메모리가 아니라 Supabase의 `messages` 테이블에 있고, 새 인스턴스도 같은 테이블을 읽기 때문입니

다. 다른 브라우저에서 앞의 대화가 보이지 않은 것은 브라우저마다 다른 게스트 사용자로 로그인하고, 서버가 모든 조회에 그 사용자의 ID 조건을 붙이기 때문입니다. 이제 두 리비전이 함께 요청을 받아도 같은 대화를 보게 되었고, 대신 두 리비전이 같은 테이블을 함께 쓰므로 한쪽이 테이블 구조를 바꾸면 다른 쪽이 바로 영향을 받습니다.

staging에서 확인했으면 production도 같은 방법으로 연결합니다. production 서비스는 실제 사용자가 쓰므로, 명령의 서비스 이름이 `friend-agent` 이고 값이 모두 production 프로젝트 것인지 한 번 더 확인합니다.

직접 실행할 명령: production 서비스에 Supabase 연결

```
gcloud run services update friend-agent --region us-central1 \
  --update-env-vars STORAGE=supabase,AUTH=supabase,SUPABASE_URL=<production 프로젝트 URL
>,SUPABASE_PUBLISHABLE_KEY=<production publishable key> \
  --update-secrets SUPABASE_SECRET_KEY=supabase-secret-production:latest
```

객관식 퀴즈

Q. Supabase에서 받은 값 가운데 secret key 만 Secret Manager에 넣고, publishable key 는 일반 환경변수로 넣었습니다. 그 이유로 가장 알맞은 것은 무엇인가요?

• 1. secret key 가 더 길어서 환경변수에 들어가지 않기 때문이다
• 2. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터에 접근할 수 있어서 서버에서만 써야 하고, publishable key 는 브라우저로 전달되는 공개 값이기 때문이다
• 3. publishable key 는 Supabase가 매일 바꾸기 때문이다
• 4. Secret Manager에는 시크릿을 하나만 넣을 수 있기 때문이다

- •

  1. secret key 가 더 길어서 환경변수에 들어가지 않기 때문이다

- •

  2. secret key 는 RLS를 건너뛰어 모든 사용자의 데이터에 접근할 수 있어서 서버에서만 써야 하고, publishable key 는 브라우저로 전달되는 공개 값이기 때문이다

- •

  3. publishable key 는 Supabase가 매일 바꾸기 때문이다

- •

  4. Secret Manager에는 시크릿을 하나만 넣을 수 있기 때문이다

Chapter04

요청 확인 스크립트

배포하는 동안 사용자가 오류를 봤는지는 새로고침 몇 번으로는 알 수 없습니다. 템플릿의 `scripts/watch_requests.py` 는 주소의 `/version` 에 0.2초마다 요청을 보내고, 응답 코드와 응답한 `revision` 을 기록합니다. 연결이 실패하거나 2초 안에 답이 없어도 실패로 셉니다. `--seconds` 로 정한 시간(최대 5분)이 지나거나 Ctrl+C로 멈추면, 전체 요청 수, 실패 수, 리비전별 응답 수, 실패가 난 시각을 요약해 보여 줍니다. `/version` 은 LLM API를 부르지 않으므로 오래 돌려도 비용이 들지 않습니다.

터미널을 두 개 엽니다. 한쪽에서 staging 주소로 스크립트를 켜 두고, 다른 쪽에서 배포 명령을 실행합니다. 스크립트를 켜기 전에 브라우저로 staging 주소를 한 번 열어 둡니다. staging은 요청이 없으면 인스턴스가 0개로 줄어 있어서, 쉬고 있던 서비스의 첫 요청은 인스턴스를 새로 켜느라 늦어지는 콜드 스타트(cold start)를 겪고, 2초를 넘기면 배포와 상관없이 실패로 찍힐 수 있습니다. 새 리비전은 이미지를 새로 만들지 않고 화면 문구 환경변수 `BANNER` 만 바꿔 만듭니다. 배포용 이미지는 GitHub Actions에서만 만든다는 규칙을 지키면서, `/version` 의 `banner` 값으로 어느 리비전이 답했는지도 구별할 수 있습니다.

직접 실행할 명령: 기본 배포 측정 (staging)

```
# 첫 번째 터미널: 저장소 루트, 가상환경을 켠 터미널에서
python scripts/watch_requests.py https://<staging 서비스 주소> --seconds 120

# 두 번째 터미널: BANNER 만 바꾼 새 리비전 (바로 100%를 받음)
gcloud run services update friend-agent-staging --region us-central1 --update-env-vars
BANNER=측정1
```

결과 해석: 기본 배포의 실패 수

요약에 이전 리비전과 새 리비전의 응답 수가 함께 찍히고 실패는 0건이거나 아주 적습니다. 새 리비전이 포트를 연 뒤에야 트래픽이 옮겨 가고, 이전 리비전은 종료 신호를 받은 뒤 처리하던 요청을 마치기 때문입니다. 전환 직후 몇 번은 응답이 느려질 수 있는데, 새 리비전의 인스턴스가 첫 요청을 받는 콜드 스타트 때문입니다. 실패가 있다면 실패 시각을 배포 명령이 끝난 시각과 맞춰 보고, 배포와 상관없이 드문드문 났다면 내 네트워크 문제일 수 있으니 한 번 더 측정합니다.

준비가 늦은 버전

이번에는 준비가 늦은 버전을 배포합니다. 템플릿의 시작 지연 스위치 `SLOW_START_SECONDS=30` 을 켜면 앱이 포트는 바로 열지만 30초 동안 모든 요청에 503(준비 중)을 돌려줍니다. AI 서비스가 시작할 때 모델 설정이나 문서 색인을 불러오느라 준비가 늦는 상황을 흉내 낸 것입니다. 섹션 2의 시각화 ① 탭에서 본 상황을 실제 서비스로 확인합니다.

직접 실행할 명령: 준비가 늦은 버전 측정 (staging)

```
# 첫 번째 터미널
python scripts/watch_requests.py https://<staging 서비스 주소> --seconds 120

# 두 번째 터미널
gcloud run services update friend-agent-staging --region us-central1 --update-env-vars
SLOW_START_SECONDS=30,BANNER=측정2
```

결과 해석: 포트만 보는 스타트업 프로브

이번에는 새 리비전으로 바뀐 직후부터 30초 가까이 실패가 찍힙니다. 기본 스타트업 프로브는 포트가 열렸는지만 보므로 Cloud Run은 새 리비전이 준비됐다고 보고 트래픽을 옮겼고, 앱은 그 뒤 30초 동안 503을 돌려줬기 때문입니다. 사용자라면 배포 직후 30초 동안 보낸 메시지가 모두 실패한 것입니다. 배포 명령은 성공으로 끝났다는 점도 봅니다. 배포가 성공했다는 것은 Cloud Run이 정한 준비 조건을 통과했다는 뜻일 뿐이고, 사용자가 오류를 보지 않았다는 뜻은 아닙니다.

스타트업 프로브를 /health로

스타트업 프로브가 `/health` 에 HTTP로 물어보게 바꿉니다. 시작 지연 중에는 `/health` 도 503을 돌려주므로, 앱이 준비를 마칠 때까지 Cloud Run은 새 리비전을 준비 완료로 보지 않고 트래픽을 이전 리비전에 둡니다. 아래 설정은 5초마다 묻고, 12번(60초)까지 실패를 기다립니다. 60초 안에 준비되지 않으면 새 리비전은 실패로 끝나고 트래픽은 옮겨 가지 않습니다.

직접 실행할 명령: 스타트업 프로브를 바꾸고 다시 측정 (staging)

```
# 첫 번째 터미널
python scripts/watch_requests.py https://<staging 서비스 주소> --seconds 150

# 두 번째 터미널: /health 스타트업 프로브. 시작 지연은 켜 둔 채 새 리비전을 만든다
gcloud run services update friend-agent-staging --region us-central1 \
  --startup-probe=httpGet.path=/health,periodSeconds=5,failureThreshold=12,timeoutSeco
```

```
nds=2 \
  --update-env-vars BANNER=측정3
```

코딩 에이전트에게 맡길 때: 스타트업 프로브 바꾸기

```
friend-agent-staging 서비스의 스타트업 프로브를 /health 에 HTTP로 묻도록 바꿔 줘. 5초 간격, 실패 12번까지, 응답 대기 2초.
같은 명령에서 BANNER=측정3 으로 바꿔 새 리비전이 생기게 하고, SLOW_START_SECONDS 는 그대로 둬. 다른 설정은 바꾸지 마.
```

결과 해석: 준비된 뒤에 옮겨 간 트래픽

이번에는 시작 지연이 켜져 있는데도 실패가 0건입니다. 대신 배포 명령이 30초쯤 더 걸리고, 스크립트 기록에서 이전 리비전이 그만큼 더 오래 답한 뒤 새 리비전으로 바뀝니다. 콘솔의 리비전 상세 > 컨테이너 탭에서 스타트업 프로브가 HTTP `/health` 로 보이는지도 확인합니다. 실패를 없앤 대가는 배포가 끝나는 시간이고, 사용자가 오류를 보는 것보다 배포가 30초 늦어지는 편이 낫다는 판단입니다.

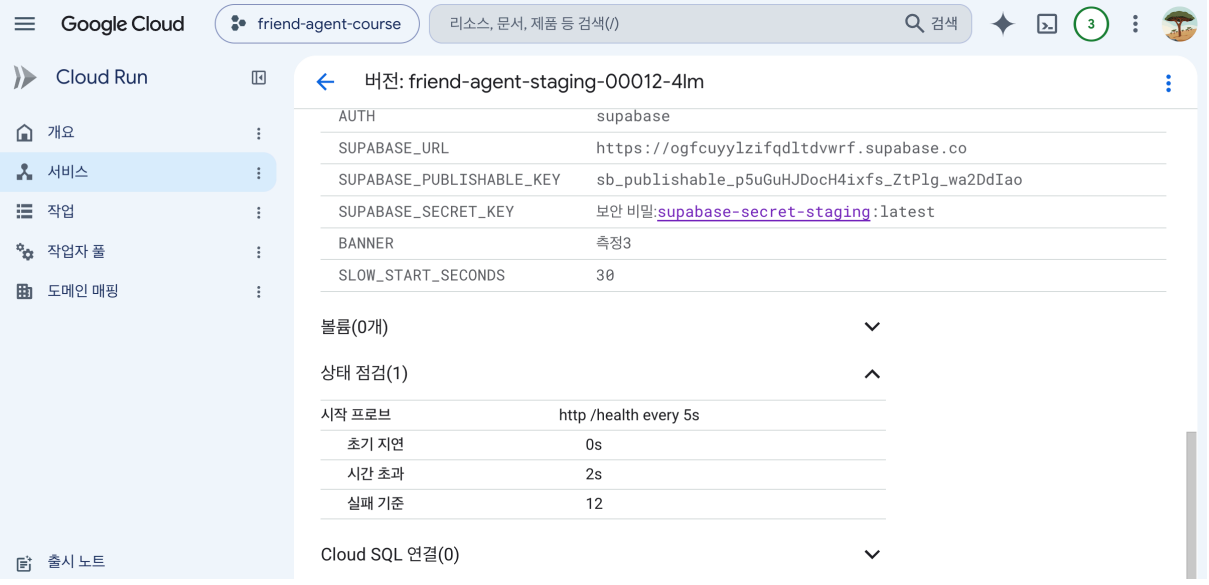


그림: Cloud Run > friend-agent-staging > 리비전 상세 > 컨테이너에서 스타트업 프로브가 HTTP /health, 5초 간격으로 설정된 모습

다음 자동 배포에도 남기기

지금은 staging 서비스에 명령으로 직접 넣었습니다. 자동 배포에서도 매번 같은 설정이 들어가게 `deploy.yml` 의 배포 단계 `flags` 줄에 같은 설정을 적어 둡니다. 그러면 production

도 다음 배포부터 같은 스타트업 프로브를 씁니다. 측정을 마쳤으면 staging의 시작 지연 스위치와 실습용 문구를 뺍니다.

직접 실행할 명령: 스타트업 프로브를 워크플로에 적고 staging 정리

```
# .github/workflows/deploy.yml 의 배포 단계 flags 줄
#   flags: --service-account=${{ vars.RUNTIME_SA }} --startup-probe=httpGet.path=/heal
th,periodSeconds=5,failureThreshold=12,timeoutSeconds=2

# 시작 지연과 실습용 문구 빼기 (새 리비전이 생기고 바로 요청을 받음)
gcloud run services update friend-agent-staging --region us-central1 --remove-env-vars
SLOW_START_SECONDS,BANNER
```

코딩 에이전트에게 맡길 때: 스타트업 프로브를 워크플로에 적고 staging 정리

```
작업 브랜치를 만들고 .github/workflows/deploy.yml 의 배포 단계 flags 줄 끝에
--startup-probe=httpGet.path=/health,periodSeconds=5,failureThreshold=12,timeoutSecond
s=2 를 붙여 줘.
--service-account 는 그대로 두고, flags 줄 말고 다른 줄과 액션 버전은 바꾸지 마. 바꾸기 전과 후의 fl
ags 줄을 나란히 보여 줘.
그다음 friend-agent-staging 서비스에서 환경변수 SLOW_START_SECONDS 와 BANNER 만 빼 줘. PR은 내
가 올린다.
```

`deploy.yml` 은 작업 브랜치에서 고치고 PR로 올려 staging을 거쳐 main까지 머지합니다. 바꾸는 줄이 `flags` 한 줄뿐인지, `--service-account` 를 지우지 않았는지 봅니다.

객관식 퀴즈

Q. 스타트업 프로브를 `/health` 로 바꾼 뒤 시작 지연 30초인 버전을 배포했더니 실패는 0건이었지만 배포 명령이 30초 넘게 걸렸습니다. 그 30초 동안 사용자 요청은 어디서 처리되었나요?

- 1. 새 리비전이 503을 돌려주며 처리했다
- 2. Cloud Run이 요청을 모아 두었다가 새 리비전이 준비된 뒤 보냈다
- 3. 트래픽이 아직 옮겨 가지 않아 이전 리비전이 처리했다
- 4. 태그 URL로 보내졌다

- •

  1. 새 리비전이 503을 돌려주며 처리했다

- •

  2. Cloud Run이 요청을 모아 두었다가 새 리비전이 준비된 뒤 보냈다

- •

3. 트래픽이 아직 옮겨 가지 않아 이전 리비전이 처리했다

- •

4. 태그 URL로 보내졌다

Chapter05

트래픽 0%로 새 리비전 올리기

staging에서 블루그린을 손으로 한 번 합니다. 먼저 지금 요청을 받는 리비전 이름을 적어 둡니다. 롤백할 때 되돌아갈 곳입니다. 그다음 `BANNER` 만 바꾼 새 리비전을 트래픽 0%로 올리고 `green` 태그를 붙입니다. 명령이 끝나면 태그 URL로 새 리비전만 열어 볼 수 있고, 원래 주소는 여전히 이전 리비전이 답합니다.

직접 실행할 명령: 지금 리비전 확인하고 0%로 새 리비전 올리기

```
# 지금 트래픽을 받는 리비전과 비율
gcloud run services describe friend-agent-staging --region us-central1 --format "yaml
(status.traffic)"

# --no-traffic: 새 리비전에 트래픽을 보내지 않음  --tag: 새 리비전에만 닿는 태그 URL을 만듦
gcloud run services update friend-agent-staging --region us-central1 --update-env-vars
"BANNER=GREEN" --no-traffic --tag green
```

코딩 에이전트에게 맡길 때: 0%로 새 리비전 올리기

```
friend-agent-staging 서비스에서 지금 트래픽을 받는 리비전 이름과 비율을 먼저 보여 줘.
그다음 환경변수 BANNER=GREEN 을 넣은 새 리비전을 만들되 트래픽은 보내지 말고 green 태그를 붙여 줘. pro
duction 과 다른 설정은 건드리지 마.
끝나면 태그 URL을 알려 줘.
```

실행 전에 서비스 이름이 `friend-agent-staging` 인지, 명령에 `--no-traffic` 이 들어 있는지 봅니다. 이 옵션이 빠지면 새 리비전이 바로 트래픽 100%를 받아 기본 배포가 됩니다.

직접 확인할 곳: 태그 URL로 새 리비전만 확인

1. 태그 URL 끝에 `/version` 을 붙여 열고 `banner` 가 `GREEN` 인지, 원래 주소의 `/version` 은 이전 값인지 비교합니다.
2. 태그 URL로 화면을 열어 메시지 하나를 보내 봅니다. 태그 URL은 서비스 주소와 호스트 이름이 달라 브라우저가 로그인 정보를 따로 보관하므로, 새 게스트 사용자로 시작합

니다. 원래 주소에서 나눈 대화가 보이지 않는 것은 이 때문이고, 데이터베이스가 달라서가 아닙니다.

3. Cloud Run > friend-agent-staging > 리비전 탭에서 새 리비전이 트래픽 0%이고 `green` 태그가 붙어 있는지 봅니다.

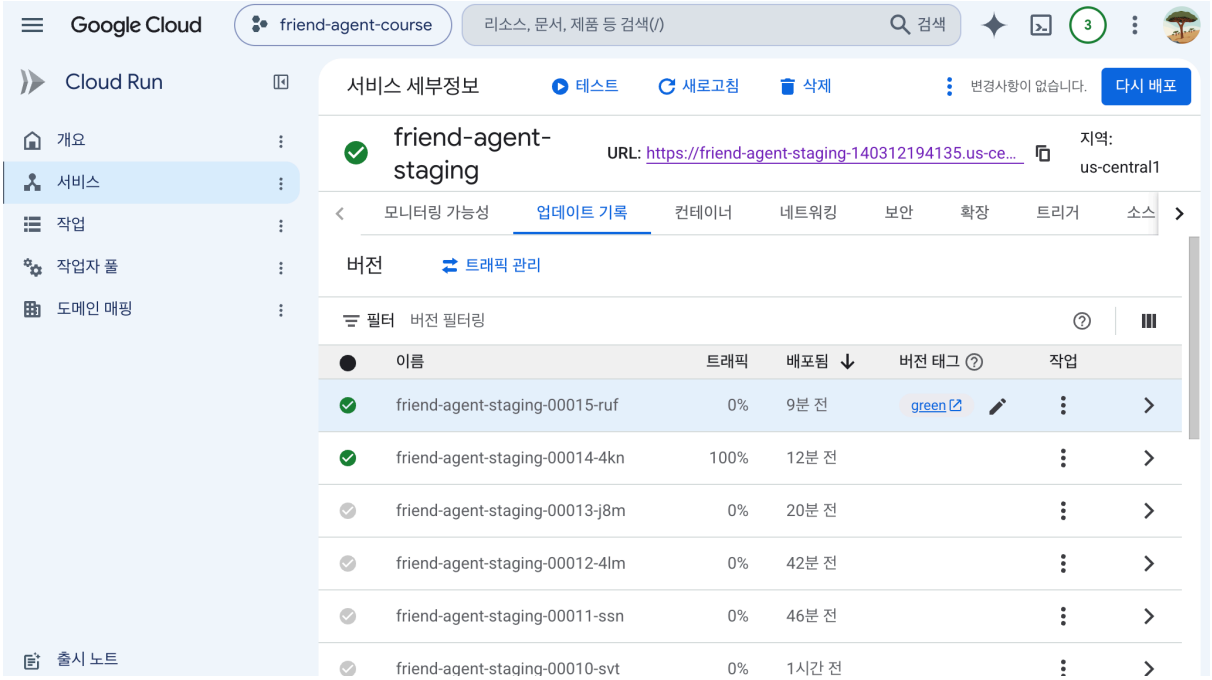


그림: Cloud Run > friend-agent-staging > 리비전 탭에서 새 리비전 트래픽 0%이고 green 태그가 붙어 있으며, 이전 리비전이 100%인 모습

한 번에 전환하고 롤백하기

요청 확인 스크립트를 켜 두고 트래픽 100%를 `green` 으로 옮깁니다. 그다음 문제가 생겼다고 가정하고 처음에 적어 둔 리비전으로 되돌립니다. 버튼을 누르듯 명령 하나로 끝나는 시간을, 코드를 되돌려 다시 빌드하고 배포하는 시간과 비교해 봅니다.

직접 실행할 명령: 전환과 롤백 (staging)

```
# 첫 번째 터미널
python scripts/watch_requests.py https://<staging 서비스 주소> --seconds 120

# 두 번째 터미널: green 으로 100%
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-tags
green=100

# 롤백: 처음에 적어 둔 리비전으로 100%
```

```
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-revi
sions <처음 리비전 이름>=100
```

결과 해석: 트래픽 설정만 바꾼 전환

요약에 두 리비전의 응답이 함께 찍히고 실패는 0건입니다. 전환과 롤백 모두 새 인스턴스를 켜거나 이미지를 만들지 않고 트래픽 설정만 바꾸므로 몇 초 안에 끝납니다. 같은 브라우저 탭을 새로고침하면 배너가 GREEN이 되었다가 롤백 뒤 사라지고, 대화는 그대로 남습니다. 두 리비전이 같은 Supabase 테이블을 보기 때문입니다. 롤백이 빨랐던 이유는 이전 리비전을 지우지 않고 남겨 두었기 때문입니다.

카나리로 나눠 보내기

이번에는 `green` 리비전으로 트래픽을 10%, 50%, 100% 순서로 나눠 옮깁니다. 10%에서 1~2분 머물며 스크립트의 리비전별 응답 수를 보고, 문제가 없으면 다음 비율로 올립니다.

직접 실행할 명령: 트래픽 비율 바꾸기와 측정

```
# 첫 번째 터미널 (최대 5분, LLM API 호출 없음)
python scripts/watch_requests.py https://<staging 서비스 주소>

# 두 번째 터미널
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-tags
green=10
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-tags
green=50
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-tags
green=100
```

코딩 에이전트에게 맡길 때: 트래픽 비율 바꾸기

```
friend-agent-staging 서비스에서 green 태그가 붙은 리비전으로 트래픽 10%를 보내 줘. 나머지 90%는 지
금 리비전에 그대로 둬.
다음 단계는 내가 숫자를 확인하고 말할 때만 진행해 줘. 비율을 바꾸는 것 말고 다른 설정은 바꾸지 마.
```

결과 해석: 요청 단위로 나뉜 비율

10%로 두면 요약에서 새 리비전 응답이 8%나 13%처럼 나옵니다. 비율은 요청 하나하나에 확률로 적용되므로 요청 수가 적을수록 설정값에서 멀어집니다. 같은 사람의 요청도 요청마다 다른 리비전으로 갈 수 있으므로, 같은 탭에서 새로고침을 여러 번 하면 배너가 나타났다

사라집니다. Cloud Run 서비스의 측정항목 탭에 있는 요청 수 그래프는 응답 코드(2xx, 4xx, 5xx)별로만 나뉩니다. 리비전별로 보려면 Cloud Monitoring의 Metrics Explorer에서 Cloud Run Revision의 Request Count를 고르고, `service_name` 으로 서비스를 거른 뒤 `revision_name` 으로 묶습니다. staging에서는 스크립트가 보낸 요청이 거의 전부라 숫자가 작습니다. production이라면 이 화면이 판단의 근거가 됩니다.

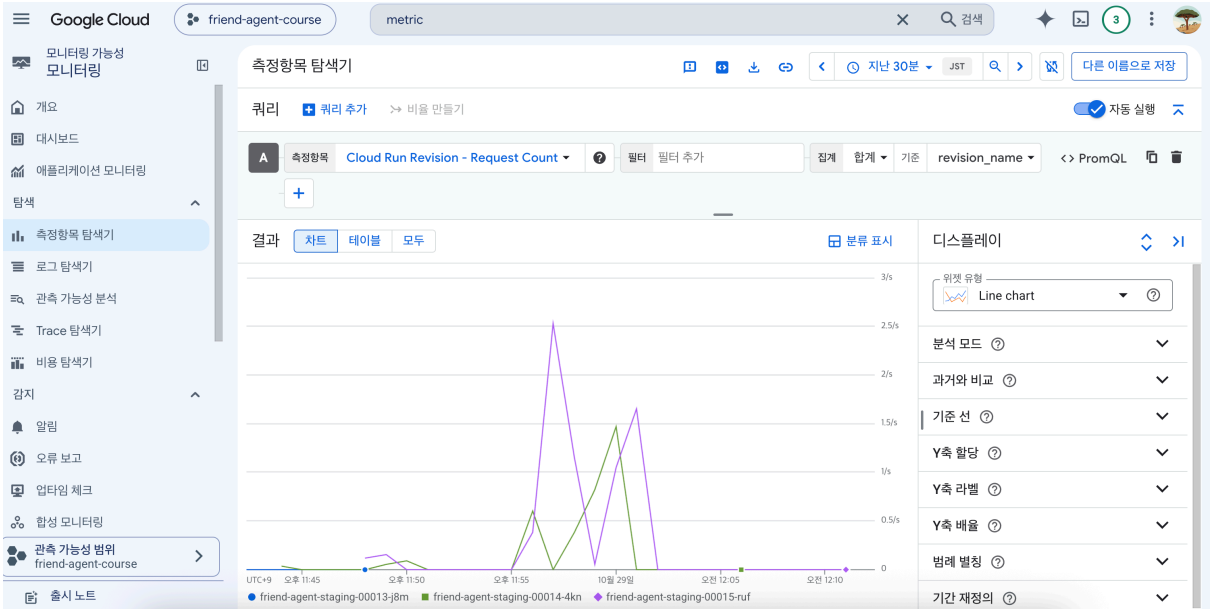


그림: Monitoring > Metrics Explorer, Cloud Run Revision > Request Count, revision_name 으로 묶음

트래픽 고정 확인하고 풀기

지금 트래픽은 `green` 태그에 정해져 있습니다. 이 상태에서 새 리비전을 하나 더 만들어, 섹션 2에서 본 트래픽 고정을 실제로 확인합니다. 새 리비전이 0%로 남는 것을 본 뒤 `--to-latest` 로 풀고, 다 쓴 태그와 실습용 문구를 지웁니다.

직접 실행할 명령: 트래픽 고정 확인과 staging 정리

```
# 고정된 상태에서 새 리비전 만들기
gcloud run services update friend-agent-staging --region us-central1 --update-env-vars
BANNER=고정확인
gcloud run services describe friend-agent-staging --region us-central1 --format "yaml
(status.traffic)"

# 가장 최근 리비전을 따라가게 하고, 태그와 실습용 문구 지우기
gcloud run services update-traffic friend-agent-staging --region us-central1 --to-late
st
gcloud run services update-traffic friend-agent-staging --region us-central1 --remove-
tags green
```

```
gcloud run services update friend-agent-staging --region us-central1 --remove-env-vars
BANNER
```

결과 해석: 배포는 끝났는데 트래픽은 그대로

`BANNER=고정확인` 으로 만든 리비전은 명령이 성공으로 끝났는데도 트래픽 0%로 남고, 서비스 주소는 여전히 `green` 리비전이 답합니다. 트래픽을 태그에 정해 둔 설정이 남아 있기 때문입니다. 자동 배포가 이 상태에서 돌면 워크플로는 초록색으로 끝나도 사용자는 새 버전을 받지 못합니다. `--to-latest` 를 실행한 뒤 다시 조회하면 트래픽이 가장 최근 리비전 100%로 바뀝니다. 오늘 후반부에 전환과 롤백을 자동화할 때 이 동작을 고려해야 합니다.

롤백을 결정하는 신호

카나리 비율을 올릴지, 지금 되돌릴지는 배포하기 전에 정해 둡니다. 숫자가 나쁘게 나온 뒤에 기준을 정하면 "조금만 더 지켜보자"는 쪽으로 기울기 쉽고, 그사이 새 버전을 경험하는 사용자가 늘어납니다. 실무에서 롤백을 결정하는 신호는 보통 아래 다섯 가지이고, 모두 이전 리비전과 나란히 비교합니다.

신호	무엇을 보나	롤백 기준의 예
오류율	새 리비전 요청 가운데 5xx 응답의 비율	이전 리비전보다 눈에 띄게 높아짐(예: 1%를 넘거나 이전의 두 배 이상)
지연 시간	응답이 느린 쪽의 지연 시간(p95, 요청 100개 중 95번째로 빠른 값)	이전 리비전보다 크게 늘어남(예: 1초 이상)
핵심 기능	로그인, 메시지 전송처럼 서비스가 꼭 해야 하는 일과 스모크 테스트	한 번이라도 실패
인스턴스 상태	시작 실패, 메모리 초과로 인한 재시작, 인스턴스 수가 갑자기 늘어남	이전 리비전에 없던 일이 생김
사용자와 비용	사용자 신고, LLM API 호출 수와 토큰 비용	새 버전 이후 갑자기 늘어남

평균이나 p50(가운데 값)만 보면 일부 요청이 아주 느려진 것을 놓치므로, 지연 시간은 느린 쪽(p95, p99)을 봅니다. 그리고 기준을 판단하려면 새 리비전이 충분한 요청을 받아야 합니다. 10%에서 몇 분 머무는 동안 새 리비전이 받은 요청이 몇 건뿐이라면, 실패 0건은 문제가 없다는 증거가 되지 못합니다. 신호가 애매할 때는 먼저 롤백하고 원인을 찾는 편이 안전합니다. 롤백은 트래픽 설정만 바꾸므로 몇 초 만에 끝나고, 다시 전환하는 것도 같은 명령 하나이기 때문입니다. 다만 새 버전이 테이블 구조를 바꿨다면 이전 리비전이 바뀐 테이블을 읽지 못할 수 있어, 이 경우에는 롤백도 미리 시험해 둔 순서로만 합니다.

배포 방식을 고르는 기준

기본 배포, 블루그린, 카나리 가운데 어느 것이 맞는지는 서비스마다 다릅니다. 아래 기준을 차례로 따져 봅니다.

기준	이럴 때	맞는 방식
요청 수	하루 요청이 수십 건 정도라 일부만 보내면 숫자가 쌓이지 않음	블루그린. 태그 URL로 직접 확인한 뒤 한 번에 전환
문제가 드러나는 방식	특정 입력이나 일부 사용자에게서만 문제가 드러남	카나리. 일부 요청으로 먼저 확인
두 버전을 함께 돌려도 되는가	새 버전과 이전 버전이 같은 테이블이나 같은 API를 다르게 쓰면 함께 돌 때 깨짐	블루그린(함께 도는 시간이 짧음). 카나리를 하려면 먼저 두 버전이 함께 동작하게 만듦
문제가 생겼을 때 허용되는 영향	결제처럼 일부 사용자라도 실패하면 안 됨	카나리로 영향 범위를 작게, 롤백 기준을 엄격하게
리비전별로 숫자를 볼 수 있는가	리비전별 오류율과 지연 시간을 볼 도구와 정해 둔 기준이 없음	블루그린이나 기본 배포. 카나리는 볼 숫자가 있어야 의미가 있음
변경의 크기	문구 하나처럼 작고 되돌리기 쉬운 변경	기본 배포도 충분함

사용자가 아직 적은 에이전트 서비스라면 블루그린으로 시작하고, 사용자가 늘어 일부 요청만으로도 숫자가 쌓이게 되면 카나리로 넘어가는 경우가 많습니다. 방식은 한 번 정하면 끝이 아니라, 요청 수와 관측 도구가 달라지면 다시 고릅니다. 오늘 후반부의 직접 해 보기는 사용자가 적은 지금의 에이전트 서비스에 맞춰 블루그린으로 자동화하고, 카나리를 자동으로 올리는 배포는 마지막 도전 과제로 둡니다.

객관식 퀴즈

Q. 하루 요청이 30건 정도인 서비스에서 "10%에서 10분 머물고, 실패 0건이면 50%로"라는 카나리 기준을 정했습니다. 이 기준의 가장 큰 문제는 무엇인가요?

- 1. 10분이 너무 길어서 배포가 늦어진다
- 2. 10분 동안 새 리비전이 받는 요청이 거의 없어 실패 0건이 아무것도 증명하지 못한다
- 3. 카나리는 요청이 적은 서비스에서는 명령이 실행되지 않는다
- 4. 실패 대신 지연 시간만 봐야 한다**제출**

- •

  1. 10분이 너무 길어서 배포가 늦어진다

- •

2. 10분 동안 새 리비전이 받는 요청이 거의 없어 실패 0건이 아무것도 증명하지 못한다

- •

3. 카나리는 요청이 적은 서비스에서는 명령이 실행되지 않는다

- •

4. 실패 대신 지연 시간만 봐야 한다

📚 블루그린과 카나리 참고 자료

- gcloud run services update-traffic: `-to-tags`, `-to-revisions`, `-to-latest`, `-remove-tags` 옵션의 정확한 문법이 있습니다.
- Martin Fowler: Canary Release: 카나리 배포라는 방식이 어떤 문제를 풀려고 생겼는지 짧게 정리한 글입니다(영어).

Chapter06

직접 해 보기의 규칙

이 섹션과 섹션 7이 오늘의 직접 해 보기입니다. 섹션 5에서 손으로 해 본 블루그린의 0% 배포와 전환을 이번에는 머지 한 번으로 저절로 일어나게 만듭니다. 지금부터는 정답 워크플로 파일이 없습니다. 각 스테이지에는 목표, 생각해 볼 것, 클리어 확인 방법, 접힌 힌트만 있고, 힌트에도 코드 대신 "무엇을 알아내야 하는지, 어디를 봐야 하는지"만 적혀 있습니다. 워크플로는 코딩 에이전트에게 맡겨 만들어도 되고 직접 써도 됩니다. 다만 만들어진 워크플로의 각 단계가 무엇을 하는지 내가 남에게 설명할 수 있어야 클리어입니다. 정답을 복사해 붙이면 돌아가기는 하지만, 배포 중에 멈췄을 때 어디를 고쳐야 할지 모르게 됩니다. 힌트는 접혀 있으니 30분쯤 혼자 찾아본 뒤에 엽니다.

목표와 클리어 조건

목표는 main에 PR이 머지되고 production 배포가 승인되면, 사람이 더 손대지 않아도 새 리비전이 트래픽 0%로 올라가고 스모크 테스트를 통과하면 전환되는 것입니다. 그동안 사용자는 오류를 한 번도 보지 않아야 합니다. 아래 네 가지가 모두 되면 클리어입니다.

1. PR을 main에 머지하고 배포를 승인한다. 그 뒤로 손대지 않는다.
2. 몇 분 뒤 서비스 주소의 `/version` 에 새 커밋이 보인다.
3. 배포하는 동안 요청 확인 스크립트의 실패가 0건이다.

4. 새 버전에 문제가 있으면 GitHub의 버튼 하나(Run workflow)로 이전 리비전으로 롤백한다.

1번과 2번만 되면 자동 배포입니다. 3번까지 되어야 무중단 배포이고, 4번의 원클릭 롤백이 있어야 문제가 생겼을 때 사람이 명령을 찾지 않고 바로 이전 리비전으로 돌아갈 수 있습니다. 아래 그림은 직접 해 보기가 끝났을 때의 흐름입니다.

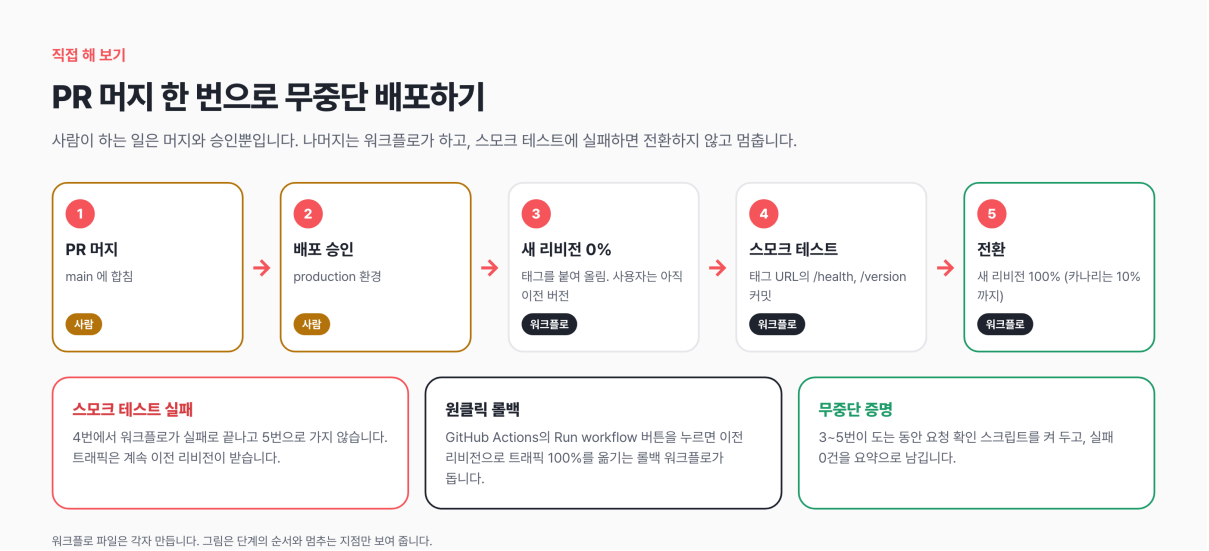


그림: PR 머지부터 전환까지의 흐름, 스모크 테스트 실패와 롤백, 무중단 증명

그림에서 사람이 하는 일은 1번과 2번뿐이고, 3-5번은 워크플로가 합니다. 지금 배포 워크플로는 이미지를 만들어 바로 100%로 배포하므로, 이번 직접 해 보기는 그 마지막 배포 단계를 3-5번으로 나누는 작업입니다. 테스트, 이미지 만들기, staging 배포, production 승인 구조는 그대로 둡니다.

스테이지 0. 손으로 한 순서 되짚기

자동화는 손으로 하던 절차를 옮겨 적는 일입니다. 옮길 원본이 없으면 워크플로도 만들 수 없습니다. 그래서 직접 해 보기를 시작하기 전에 섹션 5에서 손으로 한 블루그린 순서를 다시 훑어봅니다. 순서는 아래 다섯 단계입니다.

1. 지금 요청을 받는 리비전 이름을 확인한다(리비전 탭 또는 `gcloud run services describe`).
2. `BANNER` 만 바꾼 새 리비전을 트래픽 0%로 올리고 `green` 태그를 붙인다.
3. 태그 URL의 `/version` 과 화면으로 새 리비전만 확인한다. 원래 주소는 이전 리비전이 답하는지도 본다.

4. 요청 확인 스크립트를 켜 두고 `green` 으로 100% 옮긴다. 실패 0건과 바뀐 시각을 본다.
5. `-to-latest` 로 최신 리비전이 트래픽을 받는 상태로 돌려놓고, 태그와 `BANNER` 를 지운다.

단계마다 워크플로가 대신할 수 있는 일인지, 그때 필요한 정보(리비전 이름, 태그 URL, 이번 커밋)를 어디서 얻을지 짚어 봅니다. 섹션 5에서 실행한 명령이 그대로 워크플로의 단계가 됩니다.

생각해 볼 것: 손으로 할 때 "지금 트래픽을 받는 리비전이 무엇인지"를 콘솔에서 눈으로 확인한 순간이 있었나요? 워크플로는 콘솔을 볼 수 없습니다. 같은 정보를 명령으로 알아내려면 어떻게 해야 할까요.

스테이지 1. 머지하면 새 리비전을 0%로

배포 단계를 "새 리비전을 트래픽 0%로 올리고 태그를 붙이는" 동작으로 바꿉니다. 이 스테이지가 끝나면 머지와 승인 뒤 콘솔에 트래픽 0%인 새 리비전이 생기고, 사용자는 여전히 이전 리비전을 씁니다. 전환은 아직 하지 않습니다.

생각해 볼 것: 지금 쓰는 배포 액션에 "트래픽 없이 배포"와 "태그"에 해당하는 설정이 있을까요? 태그 이름을 배포할 때마다 같은 이름으로 붙이면, 이전 리비전에 붙어 있던 같은 이름의 태그는 어떻게 될까요? 그리고 지금 `deploy.yml` 의 배포 단계는 하나뿐이고, staging 브랜치에서는 `friend-agent-staging` 에, main에서는 `friend-agent` 에 배포하도록 staging과 production이 같이 씁니다. 이 단계를 바꾸면 두 환경이 함께 바뀝니다. 0% 배포를 두 환경 모두에 적용할지, production에만 적용할지 먼저 정합니다. 두 환경 모두에 적용하면 같은 흐름이 staging에서 먼저 돌아 보고 production에서 다시 돌고, production에만 적용하면 뒤에 넣을 스모크 테스트와 전환 단계에도 같은 조건을 계속 붙여야 합니다.

요청 틀(빈칸은 직접 채운다):

```
.github/workflows/deploy.yml 의 배포 단계를 바꿔 줘. 이 단계는 staging 과 production 이 같이 쓴다.
- 0%로 올리는 범위: [staging 과 production 둘 다 / production 만]
- 바꿀 것: 새 리비전이 트래픽을 받지 않은 채(0%) 올라오고, 태그 [        ] 가 붙게 한다.
- 바꾸지 말 것: 테스트, 이미지 만들기, production 승인 구조, 액션 버전.
- 바꾸기 전에 지금 파일의 해당 부분과 바꿀 내용을 나란히 보여 주고, 바뀌는 줄마다 무엇을 하는지 설명해 줘.
- PR은 내가 올리고 머지한다.
```

직접 확인할 곳: 스테이지 1 클리어 확인

1. 작업 브랜치에서 PR을 열기만 했을 때는 production에 아무 일도 일어나지 않는지 본다.

2. staging을 거쳐 main에 머지하고 배포를 승인한다.
3. Cloud Run > friend-agent > 리비전 탭에서 새 리비전이 트래픽 0%이고 태그가 붙어 있는지 본다.
4. 원래 서비스 주소의 `/version` 은 이전 커밋, 태그 URL의 `/version` 은 새 커밋인지 비교한다.

▼ **힌트 1**

배포 액션의 설정 목록(README의 inputs)에서 "트래픽", "태그"라는 단어가 들어간 항목을 찾아보세요. 코딩 에이전트에게는 "이 액션에서 트래픽 없이 배포하고 태그를 붙이는 설정이 무엇인지 공식 문서 근거와 함께 찾아 줘"라고 물어볼 수 있습니다. 태그는 서비스 안에서 이름이 겹칠 수 없습니다. 같은 이름을 새 리비전에 붙이면 이전 리비전에서는 어떻게 되는지 콘솔에서 두 번 배포해 보며 확인해 보세요.

스테이지 2. 전환 전 스모크 테스트

새 리비전이 "올라왔다"와 "멀쩡하다"는 다릅니다. 전환하기 전에 태그 URL의 `/health` 가 정상인지, `/version` 의 커밋이 이번에 머지한 커밋과 같은지 확인하는 단계를 넣습니다. 배포 직후 핵심 기능이 동작하는지 이렇게 빠르게 확인하는 테스트를 스모크 테스트라고 합니다. 스모크 테스트가 실패하면 워크플로는 실패로 끝나고 다음 단계(전환)로 가지 않아야 합니다.

생각해 볼 것: 원래 서비스 주소로 확인하면 누가 대답할까요? 새 리비전이 막 올라온 직후에는 첫 요청이 콜드 스타트로 늦거나 실패할 수 있는데, 몇 번까지, 얼마 간격으로 다시 물어볼까요? 스모크 테스트 단계는 태그 URL을 어디서 받아 와야 할까요?

요청 틀(빈칸은 직접 채운다):

```
.github/workflows/deploy.yml 의 배포 단계 바로 뒤에 스모크 테스트 단계를 넣어 줘.
- 확인 대상: 방금 만든 리비전의 [태그 URL / 서비스 주소] 중 [        ]
- 확인 내용: /health 가 정상 응답, /version 의 commit 이 이번 커밋과 같음
- 다시 시도: [  ]초 간격으로 최대 [  ]번
- 끝까지 실패하면: 워크플로를 실패로 끝내고 다음 단계로 가지 않게
- 다른 단계와 액션 버전은 바꾸지 마. 바뀌는 줄마다 무엇을 하는지 설명해 줘.
```

직접 확인할 곳: 스테이지 2 클리어 확인

1. 정상 버전을 머지하고 승인한다. Actions 실행 화면에서 스모크 테스트 단계가 초록색으로 끝나는지 본다.
2. 스모크 테스트 단계 로그에 어느 URL로 몇 번 물어봤는지, 받은 커밋이 무엇이었는지 보이는지 본다.

3. 이 단계까지는 리비전 탭에서 새 리비전이 여전히 트래픽 0%여야 한다.

▼ **힌트 2**

태그 URL은 서비스 주소 앞에 태그 이름과 하이픈 세 개가 붙는 형태입니다(공식 문서의 tagged revision 설명). 배포 단계가 끝나면 Actions 로그에 어떤 URL이 출력되는지 먼저 확인해 보세요. 그 URL이 태그 URL이 아니라면, 다음 단계에서 태그 URL을 어떻게 만들거나 조회할지 정해야 합니다. 한 단계의 결과를 다음 단계에서 쓰는 방법은 GitHub 문서의 단계 출력(step outputs)을 찾아보세요. 실패로 끝내는 방법은 셸 명령이 0이 아닌 값으로 끝나면 그 단계가 실패한다는 점에서 출발합니다.

주관식 퀴즈

Q. [이해 점검] 스테이지 2의 스모크 테스트를 원래 서비스 주소로 하면 어떤 일이 생길지, 새 리비전이 트래픽 0%라는 점과 연결해 설명해 보세요.

0/300**다음**

스테이지 3. 전환

스모크 테스트를 통과하면 새 리비전으로 트래픽 100%를 한 번에 옮깁니다. 블루그린 전환입니다.

생각해 볼 것: 전환할 때 리비전 이름으로 옮길까요, 태그로 옮길까요? 전환한 뒤 트래픽이 특정 리비전이나 태그에 묶여 있으면 다음 배포의 0% 단계에 어떤 영향이 있을까요? 섹션 5에서 트래픽 고정을 확인하고 풀 때 무엇을 했는지 떠올려 보세요.

요청 틀(빈칸은 직접 채운다):

```
.github/workflows/deploy.yml 의 스모크 테스트 단계 뒤에 전환 단계를 넣어 줘.
- 전환 방식: 블루그린, 새 리비전으로 100%
- 옮길 대상: [태그 이름 또는 리비전을 정하는 방법]
- 스모크 테스트 단계가 실패하면 이 단계는 실행되지 않아야 한다.
- 전환 전에 지금 트래픽을 받던 리비전 이름을 로그에 남겨 줘(롤백할 때 쓴다).
- 다른 단계와 액션 버전은 바꾸지 마. 바뀌는 줄마다 무엇을 하는지 설명해 줘.
```

직접 확인할 곳: 스테이지 3 클리어 확인

1. 머지와 승인 뒤 손대지 않고 기다린다.
2. Cloud Run > friend-agent > 리비전 탭에서 새 리비전이 100%인지 본다.
3. 서비스 주소의 `/version` 이 새 커밋인지 본다.
4. 같은 작업을 두 번 연속 해도 매번 새 리비전이 0%로 올라온 뒤 전환되는지 본다.

Chapter07

스테이지 4. 무중단 증명

자동 배포가 돌아가면 이제 무중단을 숫자로 증명합니다. 배포를 승인하기 직전에 다른 터미널에서 production 주소로 요청 확인 스크립트를 켜 두고, 전환이 끝날 때까지 실패 수를 셉니다. 스크립트는 최대 5분 뒤 스스로 멈추므로, 승인과 전환이 5분 안에 끝나지 않으면 전환 직전에 한 번 더 켭니다.

생각해 볼 것: 실패가 0이 아니면 어느 순간에 났을까요? 실패 시각을 Actions 로그의 단계 시각과 맞춰 보세요. 내 와이파이 문제도 실패로 잡힐 수 있는데, 그것을 배포 탓과 어떻게 구별할까요?

직접 실행할 명령: 배포하는 동안 측정

```
# 배포 승인 직전에 켜 둔다 (LLM API 호출 없음, 최대 5분)
python scripts/watch_requests.py https://<production 서비스 주소>
```

요약에서 전체 요청 수, 실패 수, 리비전별 응답 수를 봅니다. 리비전별 응답 수에 이전 리비전과 새 리비전이 모두 보이고 실패가 0건이면, 트래픽이 끊기지 않고 옮겨 간 것을 증명한 것입니다.

스테이지 5. 원클릭 롤백

새 버전에 문제가 있으면 GitHub 저장소의 Actions 탭에서 버튼을 눌러 이전 리비전으로 트래픽 100%를 옮기는 롤백 워크플로를 따로 만듭니다. 버튼 한 번으로 롤백한다고 해서 원클릭 롤백이라고 부릅니다. 손으로 할 때는 콘솔에서 이전 리비전을 골랐지만, 롤백 워크플로는 "이전 리비전"이 무엇인지 스스로 알아내거나 입력으로 받아야 합니다.

생각해 볼 것: 이전 리비전 이름을 어디서 알 수 있을까요? 콘솔 리비전 목록, 스테이지 3에서 로그에 남긴 이름, 버튼을 누를 때 직접 입력하는 방법이 있습니다. 이 워크플로도 Google Cloud에 접근해야 하는데, 지금 쓰는 배포 서비스 계정(배포 작업에만 쓰는 서비스 계정) 연결을 그대로 쓸 수 있을까요? 롤백에도 production 승인을 거치게 할지 정합니다. 급할 때 승인을 기다려야 하는 것과 아무나 누를 수 있는 것 가운데 어느 쪽 위험이 더 큰지 따져 보세요.

요청 틀(빈칸은 직접 채운다):

```
.github/workflows/rollback.yml 을 새로 만들어 줘.
- 실행 방법: Actions 탭의 Run workflow 버튼으로만 실행
- 버튼을 누를 때 받을 값: [되돌아갈 리비전 이름 / 받지 않고 스스로 찾기]
- 하는 일: friend-agent 서비스의 트래픽 100%를 그 리비전으로 옮기고, 끝난 뒤 /version 을 출력
- 인증: 배포 워크플로와 같은 방식, 같은 변수. production 승인은 [거친다 / 거치지 않는다]
- 액션 버전은 배포 워크플로와 같게. 파일의 줄마다 무엇을 하는지 설명해 줘.
```

직접 확인할 곳: 스테이지 5 클리어 확인

1. GitHub 저장소 > Actions > 왼쪽 목록에서 롤백 워크플로를 고르고 Run workflow 를 누른다.
2. 실행이 끝나면 Cloud Run > friend-agent > 리비전 탭에서 이전 리비전이 100%인지 본다.
3. 버튼을 누른 시각부터 `/version` 이 이전 커밋으로 바뀐 시각까지 걸린 시간을 잰다.
4. 다시 새 버전으로 돌아갈 방법(같은 버튼에 새 리비전 이름 넣기 등)도 한 번 해 본다.

▼ **힌트 5**

버튼으로 실행하는 워크플로는 GitHub 문서의 workflow_dispatch 이벤트를 찾아보세요. 입력 칸을 만드는 방법도 같은 문서에 있습니다. 권한 오류가 나면 오류 메시지에 나온 권한 이름을 먼저 읽고, 배포 서비스 계정에 그 권한 하나만 더할지, 지금 역할로 되는 다른 방법이 있는지 확인한 뒤 결정하세요. 넓은 역할을 주는 지름길은 승인하지 않습니다.

스테이지 6. 망가진 버전으로 시험

스모크 테스트 단계가 정말 전환을 막는지 시험합니다. `/health` 가 항상 실패하도록 바꾼 PR을 main에 머지하고 승인합니다. 워크플로는 실패로 끝나야 하고, 서비스 주소는 계속 이전 버전으로 답해야 합니다. 이 시험을 하는 동안에도 요청 확인 스크립트를 켜 두면, 사용자 쪽에서는 아무 일도 없었다는 것을 숫자로 남길 수 있습니다.

어느 단계에서 멈췄는지도 봅니다. 스타트업 프로브를 `/health` 로 설정해 두었다면 Cloud Run이 새 리비전을 준비 완료로 보지 않아 배포 단계에서 먼저 멈출 수 있고, 그렇지 않다면 스테이지 2의 스모크 테스트 단계에서 멈춥니다. 어느 쪽이든 전환 단계까지 가지 않았으면 클리어입니다. 확인한 뒤에는 GitHub의 Revert 버튼으로 revert 커밋을 만들어 망가진 변경을 되돌립니다.

직접 확인할 곳: 스테이지 6 클리어 확인

1. Actions 실행 화면에서 실패한 단계 이름과, 전환 단계가 실행되지 않고 건너뛰어졌는지 본다.

2. Cloud Run > friend-agent > 리비전 탭에서 트래픽 100%가 여전히 이전 리비전인지 본다.
3. 요청 확인 스크립트 요약의 실패 수가 0건인지 본다.
4. Revert PR을 머지한 뒤 정상 배포가 다시 끝까지 도는지 본다.

객관식 퀴즈

Q. `/health` 가 항상 실패하는 버전을 머지했는데, 워크플로가 초록색으로 끝나고 서비스 주소의 `/version` 이 새 커밋으로 바뀌었습니다. 가장 먼저 의심할 곳은 어디인가요?

• 1. 요청 확인 스크립트의 간격이 너무 길었다
• 2. 스모크 테스트 단계가 원래 서비스 주소를 보았거나, 스모크 테스트 실패가 단계 실패로 이어지지 않았다
• 3. Cloud Run의 최대 인스턴스 수가 부족했다
• 4. production 승인을 너무 빨리 눌렀다**제출**

- •

  1. 요청 확인 스크립트의 간격이 너무 길었다

- •

  2. 스모크 테스트 단계가 원래 서비스 주소를 보았거나, 스모크 테스트 실패가 단계 실패로 이어지지 않았다

- •

  3. Cloud Run의 최대 인스턴스 수가 부족했다

- •

  4. production 승인을 너무 빨리 눌렀다

단계별 선택 과제

스테이지 6까지 끝났다면 아래에서 하나 이상을 골라 해 봅니다. 기본은 오늘 만든 워크플로를 조금 바꾸는 과제이고, 도전은 블루그린으로 만든 흐름을 카나리로 넓혀 섹션 5의 롤백 신호를 워크플로가 직접 판단하게 하는 과제입니다. 어느 과제든 무엇을 바꾸고 무엇으로 확인할지 먼저 정하고, 워크플로의 다른 부분은 건드리지 않습니다. 앞의 스테이지와 마찬가지로 정답 파일은 없습니다.

🚀 더 해보기: 머지 한 번 배포를 내 손으로 넓히기

1. 기본: 배포 결과를 Discord나 Slack으로 알리기

- 접근 힌트: 배포 성공, 실패, 롤백마다 메시지를 보내는 단계를 워크플로 끝에 붙인다. 웹훅 URL은 시크릿(secret, API 키처럼 밖에 드러나면 안 되는 값)이므로 GitHub Secrets에 넣고 워크플로 파일에는 이름만 쓴다
- 성공하면: 망가진 버전을 한 번 더 머지했을 때 실패 알림이 오고, 워크플로 파일 어디에도 웹훅 URL이 보이지 않는다

2. 도전: 카나리 자동 승격 배포

- 접근 힌트: 10%로 전환한 뒤 정한 시간만큼 기다리고, 섹션 5의 롤백 신호(오류율, 지연 시간)를 숫자로 확인해 통과하면 50%, 100%로 올린다. 통과하지 못하면 이전 리비전으로 롤백하고 워크플로를 실패로 끝낸다
- 성공하면: 요청 확인 스크립트의 리비전별 응답 수가 10%, 50%, 100% 순서로 바뀌고, 망가진 버전은 10%에서 롤백된다

과제를 코딩 에이전트에게 맡길 때는 아래 틀의 빈칸을 내가 먼저 채웁니다. 무엇을 언제 어떻게 판단할지는 내가 정하고, 에이전트는 그 구조대로 워크플로를 고치게 합니다.

```
.github/workflows/deploy.yml 에 [기본: 알림 단계 / 도전: 카나리 자동 승격]을 추가하려고 해.
조건: [언제 무엇을 보낼지 / 몇 분 기다리고 어떤 숫자를 넘으면 되돌릴지]
시크릿은 [GitHub Secrets 이름]에서 읽고, 다른 단계와 액션 버전은 바꾸지 마.
바꾸기 전에 어느 줄을 왜 바꾸는지 먼저 설명해 줘.
```

생각해 보기: 스키마 변경이 있는 배포

에이전트 서비스는 이제 Supabase 데이터베이스에 저장하고, 템플릿의 `supabase/migrations/0001_init.sql` 에 적힌 테이블(`messages`, `memories`, `reminders` 등)을 씁니다. 이 파일을 열어 보고, 오늘 만든 배포 흐름에서 새 버전이 `reminders` 테이블의 열 이름 하나를 바꾸는 스키마 마이그레이션(schema migration, 테이블 구조를 바꾸는 작업)을 한다면 어떤 일이 생길지 정리해 봅니다. 전환하는 동안에는 이전 리비전과 새 리비전이 같은 테이블을 함께 쓰고(카나리라면 10%에 머무는 동안 더 오래), 롤백 버튼을 누르면 이전 리비전이 바뀐 테이블을 다시 읽게 된다는 점에서 출발합니다.

서로 보여 주기

옆 사람과 짝을 지어 5분씩 내 배포 흐름을 보여 줍니다. PR 하나를 머지하고 승인한 뒤, 콘솔의 트래픽 비율이 바뀌는 모습과 요청 확인 스크립트의 요약을 함께 봅니다. 듣는 사람은 "스모크 테스트가 실패하면 어디서 멈추는가", "롤백 버튼은 어느 리비전으로 돌아가는가" 두 가지를 물어봅니다. 고른 선택 과제가 있으면 그 결과도 함께 보여 줍니다.

자가 점검

- [ ] 워크플로 파일에 API 키, 서비스 계정 키 같은 시크릿이 없다
- [ ] PR을 열기만 했을 때는 production에 배포되지 않는다
- [ ] 스모크 테스트가 실패하면 전환되지 않는다
- [ ] 두 번 연속 배포해도 매번 새 리비전이 0%로 올라온 뒤 전환된다
- [ ] 배포 중 실패 0건을 요약으로 증명했다
- [ ] 버튼 하나로 이전 리비전으로 롤백할 수 있다
- [ ] 액션 버전이 템플릿과 같고, 넓은 역할을 새로 주지 않았다
- [ ] 워크플로의 각 단계가 무엇을 하는지 남에게 설명할 수 있다

직접 확인할 곳: 마지막 점검

1. `.github/workflows/` 의 두 파일을 열고, 단계마다 이름 옆에 하는 일을 한 문장으로 말해 본다. 막히는 단계가 있으면 그 단계의 공식 문서를 다시 읽는다.
2. Google Cloud 콘솔 > IAM 및 관리자 > IAM 에서 배포 서비스 계정의 역할이 오늘 아침과 같은지(바뀌었다면 무엇이 왜 늘었는지) 본다.

주관식 퀴즈

Q. [직접 해 보기 회고] 직접 해 보기에서 가장 오래 막힌 곳을 하나 골라, 무엇을 보고 막힌 줄 알았는지, 원인을 어디서 찾았는지, 어떻게 풀었는지 순서대로 적어 보세요.

0/300**다음**

📚 직접 해 보기 참고 자료

- google-github-actions/deploy-cloudrun: 배포 액션의 설정 목록과 출력값을 확인할 수 있습니다. 스테이지 1과 2에서 봅니다.
- Passing information between jobs: 한 단계나 작업의 결과를 다음 단계에서 쓰는 방법입니다. 스테이지 2와 3에서 봅니다.
- Events that trigger workflows: workflow_dispatch: 버튼으로 실행하는 워크플로와 입력 칸 만드는 법이 있습니다. 스테이지 5에서 봅니다.
- Manually running a workflow: Actions 탭에서 Run workflow 버튼을 찾고 누르는 화면 안내입니다.

Chapter08

오늘 배운 것

Cloud Run이 맡는 것과 내가 정하는 것. Cloud Run은 로드밸런서를 운영하고, 리비전 사이의 트래픽 설정으로 요청을 옮기고, 이전 인스턴스에 종료 신호를 보낸 뒤 정리합니다. 새 리비전이 언제 준비됐다고 볼지는 스타트업 프로브로 내가 정하고, 기본값인 포트 확인을 `/health` 로 바꾸자 준비가 늦은 버전에서도 실패가 0건이 되었습니다.

대화가 남는 서비스. staging과 production을 Supabase에 연결하고 secret key 를 시크릿으로 넣어, 새 리비전이 떠도 대화가 남고 두 리비전이 같은 데이터를 보게 했습니다. 대신 두 리비전이 같은 테이블을 함께 쓰게 되었습니다.

블루그린, 카나리, 트래픽 고정. 트래픽 0% 배포와 태그 URL로 확인한 뒤 전환하고 몇 초 만에 롤백했고, 비율을 나눠 카나리를 했습니다. 트래픽을 특정 리비전에 정해 두면 다음 배포가 트래픽을 받지 못한다는 것도 직접 확인했습니다.

손 작업을 워크플로로. 손으로 하던 0% 배포, 스모크 테스트, 전환을 워크플로의 단계로 옮겼고, 스모크 테스트가 실패하면 전환하지 않고 멈추는 것과 원클릭 롤백을 시험했습니다. 무중단은 요청 확인 스크립트의 실패 0건으로 증명했습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 한 번에 전환할 때와 10%만 보낼 때를 비교해 적었습니다. 그 답을 다시 읽고, 오늘 직접 본 것(준비가 늦은 버전의 실패, 태그 URL로 미리 본 새 리비전, 카나리 비율, 트래픽 고정, 직접 해 보기에서 멈춘 단계)과 비교해 생각이 달라진 부분과 그대로인 부분을 적어 보세요.